

Möbius-Tripolarer Mikro-Lift für Topologische Panoptische Triangulation

Formale Master-Spezifikation v1.0

Sebastian Klemm

10. Mai 2026

Zusammenfassung

Diese Spezifikation definiert den *Möbius-Tripolaren Mikro-Lift* (MTL) als implementierbares Erweiterungsprofil der Topologischen Panoptischen Triangulation. MTL ersetzt weder TPC noch TPT. Es fügt zwischen *PhaseSpaceWindow* und *MeshHolo*-Triangulation eine lokale, deterministische Mikro-Faser-Schicht ein. Jeder zulässige TPTPoint wird nur innerhalb eines lokalen Subwindows geliftet und erhält eine tripolare Mikro-Faser aus Primärphase, dualer Antiphase und Mandorla-/Seam-Komponente. Der Null-Center bleibt zustandsloser Ordnungsanker; er ist kein dritter datenhaltiger Vertex. Ziel ist nicht rohe Beschleunigung, sondern robustere lokale Disambiguierung, explizite Dualisierungsnachweise, bessere Carrier-Kohärenz und gatefähige Rückinterpretation.

Inhaltsverzeichnis

1	Status, Scope und Normativität	3
1.1	Status	3
1.2	Normative Sprache	3
1.3	Ziel	3
1.4	Nicht-Ziele	3
1.5	Quellenordnung	3
2	Kristallisierte Kernentscheidung	3
3	Formaler Kernel	4
4	Datenobjekte	4
5	Operatoren	5
5.1	Primärphase	5
5.2	Duale Antiphase	5
5.3	Seam- und Mandorla-Bildung	6
5.4	Lokaler holografischer Patch	6
6	Mikro-geliftete Triangulation	7

7 Gate-Familie	7
8 Artefakte und Datenmodelle	8
8.1 MTLBundle	8
8.2 Minimales JSON-Schema	8
9 Referenzalgorithmus	9
10 Komplexität	10
11 Evaluation	10
11.1 Ablationsprotokoll	10
12 Konformitätsklassen	11
13 Negative Tests	11
14 Referenzarchitektur	11
15 Architekturdiagramm	12
16 Coding-Agent-Auftrag	12
17 Definition of Done	13
A Mapping-Tabelle der extrahierten Kerne	13
B Schlussformel	13

1 Status, Scope und Normativität

1.1 Status

Dieses Dokument ist eine normative, deutschsprachige Master-Spezifikation. Eine Implementierung, die Konformität beansprucht, **MUST** alle Anforderungen der beanspruchten Konformitätsklasse erfüllen.

1.2 Normative Sprache

Die Schlüsselwörter **MUST**, **MUST NOT**, **SHOULD** und **MAY** sind normativ. Aussagen ohne diese Schlüsselwörter sind informativ, erläuternd oder motivierend.

1.3 Ziel

MTL definiert einen geschlossenen Operatorapparat für lokale tripolare Mikro-Fasern innerhalb von TPT. Das System soll aus einem TPT-PhaseSpaceWindow ein erweitertes, aber deterministisches triangulierbares Objekt erzeugen:

$$\Sigma_{TPC} \xrightarrow{AD} W_{H^5, K} \xrightarrow{\text{Lift}_{MTL}} W_{H^5, K}^\mu \xrightarrow{T^\mu} \mathcal{M}_H^\mu \xrightarrow{R} (H, \bar{H}, Claims, Gates) \xrightarrow{G} Bundle.$$

1.4 Nicht-Ziele

MTL ist nicht:

1. ein Ersatz für TPC oder TPT;
2. eine globale Ontologie oder physikalische Behauptung;
3. eine triviale Verdreifachung aller Mesh-Vertices;
4. ein autonomer Schwarm ohne zentrale Gate- und Trace-Logik;
5. ein Automatismus für Wahrheit, Prognose oder externe Handlung;
6. eine Kommunikations-, Maskierungs- oder Infrastrukturkompressionsarchitektur.

1.5 Quellenordnung

Normative Basis ist: TPC v2.0 für State-first-Semantik, Gates, Trace, Boundaries und Replay; TPT v3.0 für PhaseSpaceWindow, TPTPoint, orthogonale Triangulation, MeshHolo, CarrierContext und Rückinterpretation; der Deep-Research-Report für die Syntheseentscheidung zum Mikro-Lift; die Helix-Mandorla-Struktur für Null-Center, intrinsische Zeit, extrinsische Zeit, Helixpaar, Mandorla und Phase-Ladder; TriMöbius und Tripolare Logik nur als reduzierter Strukturkern für duale Zeitkopplung und tripolare Zustandssemantik.

2 Kristallisierte Kernentscheidung

Invariante 2.1 (Kernschutz). *MTL MUST TPC-Gates, TPC-Trace, TPC-Claim-Status und TPC-Boundaries vollständig erhalten. Topologische oder seam-basierte Kohärenz ist Evidenz für Struktur, nicht Wahrheit.*

Invariante 2.2 (PhaseSpaceWindow als Minimalträger). *Ein isolierter Punkt MUST NOT direkt als autonomes Tripol trianguliert werden. Die kleinste zulässige Einheit für MTL ist ein PhaseSpaceWindow oder ein daraus deterministisch abgeleitetes lokales Subwindow.*

Invariante 2.3 (Null-Center bleibt zustandslos). *Der Null-Center n_0 MUST als zustandsloser Ordnungsanker behandelt werden. Er MUST NOT als datenthaltiger Vertex, dritter Pol, Speicher oder dynamischer Zustand implementiert werden.*

Invariante 2.4 (Mandorla als dritter operativer Pol). *Die dritte operative Komponente der Mikro-Faser ist die Mandorla-/Seam-Komponente m_i , nicht n_0 .*

Invariante 2.5 (Deterministische Mikro-Dualisierung). *Jede Antiphase p_i^- MUST aus einer deklarierten Dualisierungsregel, Symmetriebindung und RunDescriptor-Version hervorgehen. Beliebiges Negieren oder nicht dokumentiertes Spiegeln ist nicht konform.*

3 Formaler Kernel

Definition 3.1 (MTL-Kernel). *Ein MTL-System ist ein Tupel*

$$\text{MTL} = (W, K, N, \Delta, \Xi, \Psi, T^\mu, J^\mu, G^\mu, R^\mu, E^\mu, \text{RD}, V),$$

mit:

Symbol	Bedeutung
W	<i>TPT-PhaseSpaceWindow</i>
K	<i>CarrierContext $(n_0, t_2, t_1, \alpha, \beta, m, \Lambda)$</i>
N	<i>deterministische Nachbarschafts- und Subwindow-Familie</i>
Δ	<i>Dualisierungsoperatorfamilie</i>
Ξ	<i>Seam-/Interferenzoperatorfamilie</i>
	<i>Mikro-Faser-Domäne</i>
Ψ	<i>lokaler holografischer Patch-Raum</i>
T^μ	<i>mikro-geliftete Triangulationsfamilie</i>
J^μ	<i>Kosten- und Bewertungsfunktion für MeshHolo$^\mu$</i>
G^μ	<i>MTL-Gate-Familie</i>
R^μ	<i>Rückinterpretation in TPC/TPT-Claims und Horizonte</i>
E^μ	<i>Artefakt- und Emissionsoperatoren</i>
RD	<i>deterministischer RunDescriptor</i>
V	<i>Evaluations-, Test- und Konformitätsprofil</i>

Definition 3.2 (Gültiger MTL-Zustand). *Ein MTL-Zustand s_μ ist gültig genau dann, wenn*

$$\text{valid}_\mu(s_\mu) \Leftrightarrow \text{valid}_{\text{TPT}}(s_\mu) \wedge \text{phase_windowed}(s_\mu) \wedge \text{carrier_tagged}(s_\mu) \wedge \text{micro_lifted}(s_\mu) \wedge \text{proof_com}$$

Requirement 3.1 (Wohlgeformtheit). *Jeder MTL-Operator Ω_i^μ MUST Wohlgeformtheit erhalten:*

$$\text{WF}(s_i) = 1 \wedge \text{Adm}_{\Omega_i^\mu}(s_i, \text{RD}) = 1 \Rightarrow \text{WF}(\Omega_i^\mu(s_i)) = 1.$$

Bei Nichtanwendbarkeit MUST ein typisiertes Failure-Objekt erzeugt werden.

4 Datenobjekte

Definition 4.1 (PhaseSpaceWindow). *MTL übernimmt das TPT-Objekt*

$$W = (\text{id}, \text{domain}, \text{input_refs}, \text{points}, \text{carrier}, \text{horizon_refs}, \text{constraints}, \text{boundary}, \text{sampling}, \text{trace}, r)$$

W ist die kleinste Einheit, aus der lokale Subwindows abgeleitet werden dürfen.

Definition 4.2 (TPTPoint). *Ein Punkt ist*

$$p_i = (x_i, \text{meta}_i, \text{prov}_i, \text{carrier}_i, \text{gates}_i), \quad x_i \in [0, 1]^5.$$

prov_i MUST mindestens SourceDigest, SegmentRef oder StateRef enthalten. Punkte ohne Provenienz MUST als NoiseCandidate ausgeschlossen werden.

Definition 4.3 (Lokales Subwindow). *Für einen Punkt $p_i \in W.\text{points}$ ist das lokale Subwindow*

$$U_i = N_k(p_i, W, \text{RD}) \subseteq W.\text{points}$$

eine deterministisch sortierte Nachbarschaft, erzeugt durch eine deklarierte Nachbarschaftspolitik N_k .

Requirement 4.1 (Nachbarschaftspolitik). *N_k MUST deterministisch sein und im RunDescriptor mindestens folgende Parameter deklarieren: Metrik, k oder Radius, Tie-Breaking-Regel, Boundary-Regel, Provenienzfiter und Version.*

Definition 4.4 (Mikro-Faser). *Die MTL-Mikro-Faser eines Punktes p_i relativ zu U_i ist*

$$\mu_i = (p_i^+, p_i^-, m_i; n_0, t_2, t_1, \lambda_i, \text{proof}_i, \text{trace}_i).$$

Dabei ist p_i^+ die lokale Primärrepräsentation, p_i^- die duale Antiphase unter MTL-Dualisierung, m_i die Seam-/Mandorla-Komponente, n_0 der zustandslose Anker, t_2 die intrinsische Evolutionskoordinate, t_1 die extrinsische Commit-/Persistenzkoordinate und λ_i die aktive oder reservierte Trägerinstanz.

Definition 4.5 (Impliziter Faser-Lift). *MTL-1 verwendet einen impliziten Lift: die Vertex-Identität von p_i bleibt erhalten; μ_i wird als Descriptor an den Vertex gebunden. Ein expliziter Lift, der p_i^+ , p_i^- und m_i als eigene Vertices materialisiert, ist erst ab MTL-3 zulässig.*

5 Operatoren

5.1 Primärphase

Definition 5.1 (Primärrepräsentation). *Die Primärphase eines Punktes ist*

$$p_i^+ = P_{\text{RD}}^+(p_i, U_i, K) = (x_i, f_i^+, \text{carrier}_i, \text{prov}_i),$$

wobei f_i^+ ein deterministisch berechneter lokaler Feature-Vektor ist. Minimal:

$$f_i^+ = (x_i, \bar{x}_{U_i}, \text{cov}_{U_i}, \rho_i, \tau_i, \chi_i).$$

5.2 Duale Antiphase

Definition 5.2 (Möbius-Dualisierung). *Eine Dualisierungsregel ist eine Funktion*

$$\Delta_{\text{RD}} : (p_i^+, U_i, K) \rightarrow p_i^-$$

mit

$$p_i^- = (D_i x_i + b_i, f_i^-, \text{carrier}_i^-, \text{prov}_i, \text{dual_proof}_i),$$

wobei D_i eine deklarierte involutive oder pseudo-involutive Transformation ist.

Requirement 5.1 (Dualisierungsvertrag). *Eine konforme Dualisierung MUST mindestens erfüllen:*

1. deterministische Ableitung aus (p_i, U_i, K, RD) ;
2. deklarierte Symmetriegruppe oder lokale Dualitätsregel;
3. bounded output: $D_i x_i + b_i \in [0, 1]^5$ oder typisiertes *DualizationFailure*;
4. Proof-Objekt mit Operator-ID, Parametern und Digest;
5. Stabilitätsprüfung unter Replay.

Definition 5.3 (Standard-Dualisierung MTL-D1). Die minimal zulässige Standard-Dualisierung ist

$$D_i = I - 2 \frac{v_i v_i^\top}{\|v_i\|^2 + \epsilon}, \quad v_i = x_i - \bar{x}_{U_i},$$

$$p_i^- = \text{clip}_{[0,1]^5}(\bar{x}_{U_i} + D_i(x_i - \bar{x}_{U_i})).$$

Sie spiegelt den Punkt an der lokalen orthogonalen Gegenrichtung relativ zum Subwindow-Zentrum. Bei $\|v_i\| < \epsilon$ MUST eine fallback-Regel verwendet oder *DualizationDegenerate* emittiert werden.

5.3 Seam- und Mandorla-Bildung

Definition 5.4 (Seam-Operator). Ein Seam-Operator ist eine deterministische Abbildung

$$\Xi_{\text{RD}} : (p_i^+, p_i^-, U_i, K) \rightarrow m_i.$$

Die Standardform ist eine bilineare, normalisierte Interferenzprojektion:

$$z_i = \begin{bmatrix} x_i^+ \\ x_i^- \\ x_i^+ \odot x_i^- \\ |x_i^+ - x_i^-| \end{bmatrix}, \quad m_i = \sigma(A_{\text{RD}} z_i + B_{\text{RD}} \phi(U_i, K)),$$

wobei σ komponentenweise nach $[0, 1]$ projiziert und ϕ lokale Carrier-, Entropie- und Topologiefeatures liefert.

Requirement 5.2 (Seam-Gültigkeit). Ein m_i ist gültig genau dann, wenn

$$\text{valid}_{\text{seam}}(m_i) \Leftrightarrow \text{bounded}(m_i) \wedge \text{symmetric_admissible}(m_i) \wedge \text{carrier_coherent}(m_i) \wedge \text{traceable}(m_i).$$

Ungültige Seams MUST als *SeamFailure* protokolliert werden.

5.4 Lokaler holografischer Patch

Definition 5.5 (Holografischer Patch). Der lokale holografische Patch eines Punktes ist

$$\psi_i = \text{Patch}(p_i, U_i, \mu_i) = (U_i, p_i^+, p_i^-, m_i, C_i, B_i, \text{proof}_i),$$

wobei C_i lokale Kohärenz-/Resonanzmaße und B_i Boundary- und Gate-Signale enthält.

Requirement 5.3 (Kein globaler Einzelpunkt-Holografismus). Ein einzelner Punkt ohne Subwindow, Provenienz, Carrier und SeamProof MUST NOT als holografischer Patch klassifiziert werden.

6 Mikro-geliftete Triangulation

Definition 6.1 (Geliftetes Window). *Aus einem PhaseSpaceWindow W entsteht*

$$W^\mu = \text{Lift}_{MTL}(W) = (W, \{\mu_i\}_{i \in I}, \{\psi_i\}_{i \in I}, \text{lift_proof}, \text{lift_trace}).$$

Definition 6.2 (Mikro-geliftetes Triangulationsprodukt).

$$T^\mu(W^\mu) = T_{scale}(W) \otimes_G T_{carrier}(W) \otimes_{RD} T_{time}(W) \otimes_{RD} T_{micro}(W^\mu).$$

T_{micro} modifiziert nicht zwingend die Vertexmenge, sondern erweitert Kanten-, Simplex- und Refinementbewertungen um Mikro-Faser-Signale.

Definition 6.3 (Mikro-Kantenbewertung). *Für eine Kante (i, j) ist*

$$w_{ij}^\mu = a_1 d(x_i, x_j) + a_2 d(m_i, m_j) + a_3 d(p_i^-, p_j^-) - a_4 S_{seam}(i, j) - a_5 C_{carrier}(i, j) + a_6 B_{boundary}(i, j),$$

wobei alle Koeffizienten und Metriken im RunDescriptor deklariert werden müssen.

Definition 6.4 (Kostenfunktion MeshHolo $^\mu$). *Für eine Kandidatentriangulation T gilt*

$$J^\mu(T) = \alpha_1 \text{Sym}(T, G) + \alpha_2 \text{Res}(T, F) + \alpha_3 \text{Seam}(T) + \alpha_4 \text{DualStab}(T) + \alpha_5 \text{Carrier}(T) - \alpha_6 \Delta \text{Betti}(T) - \alpha_7$$

MTL maximiert J^μ unter TPT-Constraints, TopologyGuard und Replay-Bindung.

Requirement 6.1 (Topologieerhaltung). *Lokale Mikro-Refinements MAY Simplizes ändern, MUST aber Betti-Vektor, Persistenzdiagramm oder erlaubte Shift-Regel dokumentieren. Unbegründete topologische Veränderung ist nicht konform.*

7 Gate-Familie

Definition 7.1 (MTL-Gates). *Die Gate-Familie ist*

$$G^\mu = (G_{prov}, G_{dual}, G_{seam}, G_{carrier}, G_{topo}, G_{entropy}, G_{replay}, G_{boundary}, G_{matrix}).$$

Gate	Bedingung
G_{prov}	Jeder geliftete Punkt hat gültige Provenienz; NoiseCandidates bleiben ausgeschlossen.
G_{dual}	Jede Antiphasse besitzt deterministischen DualProof und bounded output.
G_{seam}	Jede Mandorla-/Seam-Komponente besteht Boundedness, Symmetrie und Carrier-Kohärenz.
$G_{carrier}$	$n_0, t_2, t_1, \alpha, \beta, m, \Lambda$ werden nicht kollabiert oder semantisch vermischt.
G_{topo}	Betti-/Persistenzänderungen sind stabil oder als erlaubter Shift dokumentiert.
$G_{entropy}$	Entropie steuert Refinement/Simplification, wird aber nicht pauschal als Fehler interpretiert.
G_{replay}	Identische Inputs erzeugen identische Mikro-Fasern, Mesh-, Proof-, Trace- und Artefakt-Digests.
$G_{boundary}$	Keine Emission ohne TPC/TPT-Boundary-Passage.

Definition 7.2 (Entscheidungsfunktion).

$$Decision^\mu(W^\mu, T) = \begin{cases} EmitBundle, & \text{falls alle Gates bestehen und TPC/TPT Closure besteht,} \\ Hold, & \text{falls Struktur plausibel, aber Proof/Gate unvollständig ist,} \\ Quarantine, & \text{falls Widerspruch, Boundary-Risiko oder Trace-Lücke vorliegt,} \\ Abort, & \text{falls Wohlgeformtheit, Provenienz oder Replay verletzt ist.} \end{cases}$$

8 Artefakte und Datenmodelle

8.1 MTLBundle

Definition 8.1 (Kanonisches Artefaktbündel). *Ein MTLBundle enthält mindestens:*

1. *RunDescriptor und Operatorversionen;*
2. *SourceDigests und InputRefs;*
3. *PhaseSpaceWindow-Digest;*
4. *MicroFiberSet mit μ_i -Digests;*
5. *LocalPatchSet mit Patch-Digests;*
6. *MeshHolo $^\mu$ mit Vertices, Edges, Simplices, Invarianten und Proof;*
7. *GateReport;*
8. *TraceChain;*
9. *ReplayManifest;*
10. *Rückinterpretationsreport.*

8.2 Minimales JSON-Schema

```
{
  "mtl_version": "1.0.0",
  "run_descriptor": {"seed": "...", "operator_versions": {}, "policies": {}},
  "phase_space_window_digest": "sha256:...",
  "micro_fibers": [
    {
      "point_id": "p_i",
      "subwindow_digest": "sha256:...",
      "primary": {"vector": [0,0,0,0,0], "digest": "sha256:..."},
      "dual": {"vector": [0,0,0,0,0], "rule": "MTL-D1", "proof": "sha256:..."},
      "seam": {"vector": [0,0,0,0,0], "proof": "sha256:..."},
      "carrier": {"n0": "anchor", "t2": "...", "t1": "...", "lambda": "lambda_0"},
      "trace": "sha256:..."
    }
  ],
  "mesh_holo_mu": {
    "vertices": [], "edges": [], "simplices": [],
    "invariants": {"beti": [], "persistence": [], "entropy": 0.0},
    "proof": {"symmetry": "...", "topology_guard": true, "replay": "sha256:..."},
  },
  "gates": {"prov": true, "dual": true, "seam": true, "carrier": true,
```



```

        "topology":true, "entropy":true, "replay":true,
        "boundary":true, "matrix":true},
    "decision": "EmitBundle|Hold|Quarantine|Abort"
}

```

9 Referenzalgorithmus

```

function MTL_RUN(TPC_state, domain_profile, RD):
    require valid_TPC(TPC_state)

    W = TPT_ADAPTER(TPC_state, domain_profile, RD)
    if W is AdapterFailure:
        return Hold(W.failure_report)

    require valid_PhaseSpaceWindow(W)

    fibers = []
    patches = []
    for each p_i in canonical_sort(W.points):
        if not has_provenance(p_i):
            trace_noise_candidate(p_i)
            continue

        U_i = deterministic_subwindow(p_i, W, RD.neighborhood_policy)
        if invalid(U_i):
            emit_failure("SubwindowFailure", p_i)
            continue

        p_plus = primary_projection(p_i, U_i, W.carrier, RD)
        p_minus = dualize_mobius(p_plus, U_i, W.carrier, RD.dual_policy)
        if invalid(p_minus):
            emit_failure("DualizationFailure", p_i)
            continue

        m_i = seam_operator(p_plus, p_minus, U_i, W.carrier, RD.seam_policy)
        if invalid(m_i):
            emit_failure("SeamFailure", p_i)
            continue

        mu_i = make_micro_fiber(p_plus, p_minus, m_i, W.carrier, proof, trace)
        psi_i = make_local_patch(p_i, U_i, mu_i, proof, trace)
        fibers.append(mu_i)
        patches.append(psi_i)

    W_mu = build_lifted_window(W, fibers, patches)
    if not gate_micro_lift(W_mu):
        return Hold(gate_report(W_mu))

    mesh = triangulate_micro_lifted(W_mu, RD)
    if not topology_guard(mesh):

```

```

return Quarantine(mesh.failure_report)

report = reinterpret_mesh_to_TPC(mesh, W_mu, RD)
decision = run_all_gates(report, mesh, W_mu, RD)
return materialize_bundle(decision, W_mu, mesh, report)

```

10 Komplexität

Sei $n = |W.points|$, k die maximale Subwindow-Größe, $d = 5$ die operative Koordinatendimension und $C_T(n)$ die Basiskomplexität der verwendeten TPT-Triangulation.

Proposition 10.1 (Mikro-Lift-Kosten). *Bei deterministischer k -Nachbarschaft mit indexierter Suche kostet der Mikro-Lift*

$$C_{lift} = O(n \cdot C_N(n, k) + n \cdot k \cdot d^2 + n \cdot C_{\Xi}(k, d)).$$

Für feste kleine k, d reduziert sich dies praktisch auf $O(n \cdot C_N(n, k) + n)$.

Proposition 10.2 (Triangulationskosten). *Die Gesamtkosten sind*

$$C_{MTL} = C_{TPT}(n) + C_{lift} + C_{micro_weights} + C_{gates} + C_{replay}.$$

MTL-1 SHOULD implizite Deskriptoren verwenden, damit n nicht auf $3n$ anwächst. Explizite Vertex-Lifts erhöhen Kanten- und Simplexkosten und sind erst nach erfolgreichem MTL-1/2-Ablationsnachweis zulässig.

11 Evaluation

Definition 11.1 (Metriken). *Eine MTL-Implementierung MUST mindestens folgende Metriken berichten:*

1. **MicroLiftCoverage**: Anteil gültig gelifteter Punkte;
2. **SeamConsistencyRate**: Anteil gültiger Seam-/Mandorla-Bildungen;
3. **DualFlipStability**: Replay-Stabilität der Dualisierung;
4. **CarrierCoherence**: Kohärenz zwischen $n_0, t_2, t_1, \alpha, \beta, m, \Lambda$;
5. **TopologyRobustness**: Stabilität von Betti-/Persistenzsignaturen;
6. **FalseCrystalRate**: Anteil fälschlich kristallisierter Kandidaten;
7. **TraceCompleteness**: Anteil vollständig digestierter Operator- und Gate-Schritte.

11.1 Ablationsprotokoll

Jede Behauptung über Mehrwert MUST durch Vergleich von Baseline-TPT und TPT+MTL belegt werden. Pflichtläufe:

1. TPT ohne Mikro-Lift;
2. TPT+MTL mit Dualisierung, aber ohne Seam-Gewichtung;
3. TPT+MTL mit Seam-Gewichtung;
4. TPT+MTL mit Carrier-Gewichtung;
5. vollständiger TPT+MTL-Lauf.

12 Konformitätsklassen

Klasse	Anforderungen
MTL-0	Datenmodell, RunDescriptor, MicroFiber-Schema, Failure-Objekte, kein produktiver Mesh-Eingriff.
MTL-1	Impliziter Mikro-Faser-Lift, deterministic subwindows, Standard-Dualisierung, SeamProof, GateReport.
MTL-2	Integration in MeshHolo ^μ -Kanten-/Simplexbewertung, Topology-Guard, Ablationstest.
MTL-3	Expliziter Vertex-Lift MAY genutzt werden; erfordert Beweis, dass Topologie, Replay und Komplexitätsbudget stabil bleiben.
MTL-4	Vollständige Rückinterpretation in Claims, Horizonte, Counter-Horizonte und CrystalCandidates mit ReplayManifest.

13 Negative Tests

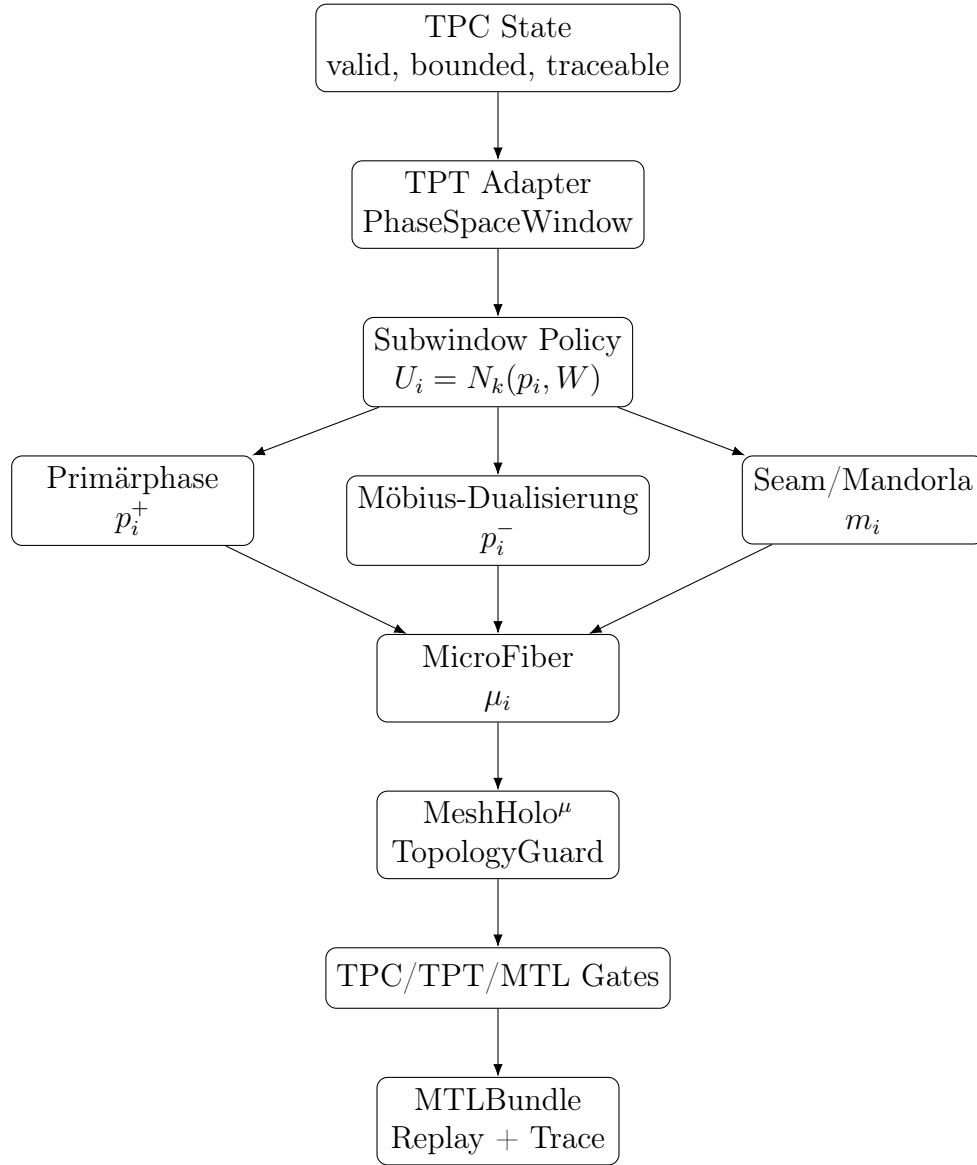
Eine konforme Implementierung MUST folgende Fälle fail-closed behandeln:

1. Punkt ohne Provenienz wird geliftet: MUST NoiseCandidate sein.
2. Null-Center wird als Vertex materialisiert: MUST Abort.
3. Antiphase entsteht durch nicht deklarierte Negation: MUST DualizationFailure.
4. Seam wird ohne Carrier-Kontext berechnet: MUST SeamFailure.
5. Identischer Input erzeugt verschiedene MicroFiber-Digests: MUST ReplayFailure.
6. Topologieänderung ohne dokumentierte Shift-Regel: MUST TopologyFailure.
7. Mesh-Kohärenz wird als Wahrheit emittiert: MUST MatrixGateFailure.

14 Referenzarchitektur

```
src/  
  tpc/                # bestehender TPC-Kern  
  tpt/                # bestehende TPT-Schicht  
    adapter/  
    carrier/  
    mesh/  
    gates/  
  mtl/  
    mod.rs            # public API  
    subwindow.rs      # deterministische lokale Nachbarschaft  
    primary.rs        # Primärprojektionen  
    dual.rs           # Möbius-Dualisierung  
    seam.rs           # Mandorla-/Seam-Operatoren  
    microfiber.rs     # MicroFiber-Datenmodell  
    patch.rs          # LocalHoloPatch  
    mesh_weights.rs   # MeshHoloμ-Bewertung  
    gates.rs          # MTL-Gates  
    replay.rs         # Digest, Trace, Replay  
    artifacts.rs      # MTLBundle JSON
```

15 Architekturdiagramm



16 Coding-Agent-Auftrag

Implementiere MTL in fünf Phasen:

1. **M0 Datenvertrag:** MicroFiber, LocalHoloPatch, LiftProof, GateReport, Failure-Klassen, JSON-Schema.
2. **M1 Impliziter Lift:** deterministische Subwindows, Primärprojektion, Standard-Dualisierung MTL-D1, Seam-Operator, Digests.
3. **M2 Gate-Integration:** ProvenanceGate, DualGate, SeamGate, CarrierGate, Replay-Gate, negative Tests.
4. **M3 Mesh-Integration:** mikro-geliftete Kanten-/Simplexkosten, TopologyGuard, MeshHolo^μ-Export.

5. **M4 Rückinterpretation:** MTL-Features in TPC/TPT-Claims, Horizonte, Gate-Signale und CrystalCandidates.

17 Definition of Done

MTL v1.0 ist implementierungsreif, wenn:

1. alle Datenmodelle kanonisch serialisierbar sind;
2. identische Inputs bitidentische Digests erzeugen;
3. jeder gültige Lift einen vollständigen Proof und Trace besitzt;
4. Null-Center niemals stateful materialisiert wird;
5. Antiphase und Seam fail-closed arbeiten;
6. TopologyGuard bei jedem Mesh-Update greift;
7. Baseline-TPT und TPT+MTL im Ablationsprotokoll vergleichbar sind;
8. MTLBundle vollständig durch Replay verifizierbar ist.

A Mapping-Tabelle der extrahierten Kerne

Quelle	Übernommener Kern	Nicht übernommen
TPC v2.0	State-first, Gates, Trace, Replay, Boundaries, Claim-Status	Domänenspezifische Deutungen
TPT v3.0	PhaseSpaceWindow, TPT-Point, orthogonale Triangulation, MeshHolo, TopologyGuard, CarrierContext	Ersatz des TPC-Kerns
Deep-Research-Report	Mikro-Lift als richtige Syntheseform, impliziter Faser-Lift, Metriken	Rohmetaphern, ungebundene Schwarmdeutung
Helix-Mandorla	n_0, t_2, t_1 , Helixpaar, Mandorla, Phase-Ladder, Migration	marktspezifische Einbettungen
TriMöbius	Nicht-orientierbare Dualisierung, intrinsisch/extrinsische Kopplung	Ledger-/Langhorizont-Produktargumente
Tripolare Logik	zwei stabile Pole plus dynamische Klasse, Resonatoridee	Hardware- und Affekt-Spezialfälle
SPEC-003r1	deterministische MeshHolo-Evolution, Symmetrie, TopologyGuard, Entropie	Router-/MEF-Spezifika
TCT, ScorpioSync	nicht benötigt	Infrastruktur- und Kommunikationsbelast

B Schlussformel

Die geschlossene Form von MTL lautet:

$$\boxed{\text{MTL} = \text{TPC}_{\text{gates,trace,replay}} + \text{TPT}_{\text{window,mesh,carrier}} + \text{Lift}_{\text{micro}}(p^+, p^-, m; n_0, t_2, t_1)}$$

MTL gewinnt seine Kohärenz nur unter vier Bedingungen: Das PhaseSpaceWindow bleibt Minimalträger; der Null-Center bleibt zustandslos; die Mandorla ist der dritte operative Pol; jede Mikro-Dualisierung ist proof-, gate-, replay- und tracepflichtig.